

Start of Baseline and Optimized ANN

Setting-up

```
# import os
# os.environ['TF_XLA_FLAGS'] = '--tf_xla_auto_jit=0'
```

```
# import tensorflow as tf
# tf.config.optimizer.set_jit(False)
```

```
!pip install scikeras==0.10.0 scikit-learn==1.5.2 tqdm kagglehub
matplotlib seaborn imblearn keras_tuner tensorflow-addons==0.18.0
```

```
Requirement already satisfied: scikeras==0.10.0 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (0.10.0)
Requirement already satisfied: scikit-learn==1.5.2 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (1.5.2)
Requirement already satisfied: tqdm in c:\users\nealb\appdata\local\
miniconda3\envs\applied_ml\lib\site-packages (4.67.3)
Requirement already satisfied: kagglehub in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (0.3.13)
Requirement already satisfied: matplotlib in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (3.9.4)
Requirement already satisfied: seaborn in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (0.13.2)
Requirement already satisfied: imblearn in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (0.0)
Requirement already satisfied: keras_tuner in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (1.4.8)
Requirement already satisfied: tensorflow-addons==0.18.0 in c:\users\
nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages
(0.18.0)
Requirement already satisfied: packaging>=0.21 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
scikeras==0.10.0) (26.0)
Requirement already satisfied: numpy>=1.19.5 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
scikit-learn==1.5.2) (1.26.4)
Requirement already satisfied: scipy>=1.6.0 in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (from scikit-
learn==1.5.2) (1.13.1)
Requirement already satisfied: joblib>=1.2.0 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
scikit-learn==1.5.2) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
scikit-learn==1.5.2) (3.6.0)
```

Requirement already satisfied: typeguard>=2.7 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from tensorflow-addons==0.18.0) (4.5.1)

Requirement already satisfied: colorama in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from tqdm) (0.4.6)

Requirement already satisfied: pyyaml in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from kagglehub) (6.0.3)

Requirement already satisfied: requests in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from kagglehub) (2.32.5)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (1.3.0)

Requirement already satisfied: cyclor>=0.10 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (4.60.2)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (1.4.7)

Requirement already satisfied: pillow>=8 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (11.3.0)

Requirement already satisfied: pyparsing>=2.3.1 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: importlib-resources>=3.2.0 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from matplotlib) (6.5.2)

Requirement already satisfied: pandas>=1.2 in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from seaborn) (2.3.3)

Requirement already satisfied: imbalanced-learn in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from imblearn) (0.12.4)

Requirement already satisfied: keras in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from keras_tuner) (2.10.0)

Requirement already satisfied: kt-legacy in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from keras_tuner) (1.0.5)

Requirement already satisfied: grpcio in c:\users\nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from keras_tuner) (1.80.0)

```

Requirement already satisfied: protobuf in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (from keras_tuner)
(3.19.6)
Requirement already satisfied: zipp>=3.1.0 in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (from importlib-
resources>=3.2.0->matplotlib) (3.23.0)
Requirement already satisfied: pytz>=2020.1 in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (from pandas>=1.2-
>seaborn) (2026.1.post1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
pandas>=1.2->seaborn) (2026.1)
Requirement already satisfied: six>=1.5 in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (from python-
dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: importlib_metadata>=3.6 in c:\users\
nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
typeguard>=2.7->tensorflow-addons==0.18.0) (8.7.0)
Requirement already satisfied: typing_extensions>=4.14.0 in c:\users\
nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
typeguard>=2.7->tensorflow-addons==0.18.0) (4.14.1)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\
nealb\appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
requests->kagglehub) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in c:\users\nealb\appdata\
local\miniconda3\envs\applied_ml\lib\site-packages (from requests-
>kagglehub) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
requests->kagglehub) (2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\nealb\
appdata\local\miniconda3\envs\applied_ml\lib\site-packages (from
requests->kagglehub) (2026.2.25)

import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

```

Loading Data

```

import kagglehub
import pandas as pd
import os

def load_dataset(data_link, file_name, target_column):
    path = kagglehub.dataset_download(data_link)
    full_path = os.path.join(path, file_name)

    try:
        df = pd.read_csv(full_path)

```

```

df = df.rename(columns={target_column: 'target'})
print("Successfully loaded dataframe.")
except Exception as e:
    print(f"Failed to load {file_name}: {e}")

return df

df_train = load_dataset("mohankrishnathalla/sleep-health-and-daily-
performance-dataset", "sleep_health_dataset.csv", "felt_rested")
# df_test = load_dataset("kaushiksuresh147/customer-segmentation",
"Test.csv", "Segmentation")

c:\Users\nealb\AppData\Local\miniconda3\envs\applied_ml\lib\site-
packages\tqdm\auto.py:21: TqdmWarning: IProgress not found. Please
update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm

Warning: Looks like you're using an outdated `kagglehub` version
(installed: 0.3.13), please consider upgrading to the latest version
(1.0.0).
Downloading from
https://www.kaggle.com/api/v1/datasets/download/mohankrishnathalla/
sleep-health-and-daily-performance-dataset?
dataset_version_number=18...

100%|██████████| 4.07M/4.07M [00:01<00:00, 2.45MB/s]

Extracting files...

Successfully loaded dataframe.

```

Initial Data Exploration

Showing First 5 Data Points

```
df_train.head()
```

	person_id	age	gender	occupation	bmi	country	\
0	1	29	Female	Driver	25.7	Japan	
1	2	55	Female	Software Engineer	22.0	USA	
2	3	42	Male	Nurse	25.0	India	
3	4	37	Female	Student	29.5	India	
4	5	23	Male	Lawyer	23.6	Spain	

	sleep_duration_hrs	sleep_quality_score	rem_percentage	\
0	6.19	6.6	22.5	
1	8.32	6.9	26.9	
2	3.74	1.0	20.2	

```

3          6.79          6.4          17.7
4          5.02          3.2          23.3

    deep_sleep_percentage ... heart_rate_resting_bpm sleep_aid_used
\
0          19.3 ...          63          0
1          14.9 ...          52          1
2          16.2 ...          72          0
3          17.7 ...          71          0
4          18.3 ...          71          0

    shift_work room_temperature_celsius weekend_sleep_diff_hrs
season \
0          0          20.1          1.84
Autumn
1          0          18.0          0.13
Winter
2          1          17.9          1.67
Spring
3          0          19.1          2.37
Summer
4          0          19.7          1.26
Summer

    day_type cognitive_performance_score sleep_disorder_risk target
0  Weekday          73.4          Healthy          0
1  Weekend          99.4          Healthy          1
2  Weekend           2.5          Severe          0
3  Weekend          67.8          Healthy          0
4  Weekday          38.1          Mild          0

[5 rows x 32 columns]

# df_test.head()

```

Identifying DataTypes and Missing Values

```

print(df_train.info())
# df_test.info()

```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   person_id                            100000 non-null  int64
1   age                                  100000 non-null  int64
2   gender                               100000 non-null  object
3   occupation                           100000 non-null  object
4   bmi                                  100000 non-null  float64
5   country                              100000 non-null  object
6   sleep_duration_hrs                   100000 non-null  float64
7   sleep_quality_score                   100000 non-null  float64
8   rem_percentage                       100000 non-null  float64
9   deep_sleep_percentage                100000 non-null  float64
10  sleep_latency_mins                   100000 non-null  int64
11  wake_episodes_per_night               100000 non-null  int64
12  caffeine_mg_before_bed                100000 non-null  int64
13  alcohol_units_before_bed              100000 non-null  float64
14  screen_time_before_bed_mins           100000 non-null  int64
15  exercise_day                          100000 non-null  int64
16  steps_that_day                        100000 non-null  int64
17  nap_duration_mins                     100000 non-null  int64
18  stress_score                          100000 non-null  float64
19  work_hours_that_day                   100000 non-null  float64
20  chronotype                            100000 non-null  object
21  mental_health_condition               100000 non-null  object
22  heart_rate_resting_bpm                100000 non-null  int64
23  sleep_aid_used                        100000 non-null  int64
24  shift_work                            100000 non-null  int64
25  room_temperature_celsius              100000 non-null  float64
26  weekend_sleep_diff_hrs                 100000 non-null  float64
27  season                                100000 non-null  object
28  day_type                              100000 non-null  object
29  cognitive_performance_score            100000 non-null  float64
30  sleep_disorder_risk                   100000 non-null  object
31  target                                100000 non-null  int64
dtypes: float64(11), int64(13), object(8)
memory usage: 24.4+ MB
None

```

Objects and Missing values are observed, maximum counts are 2627, minimum is 2358.

```

import seaborn as sns
import matplotlib.pyplot as plt

# Statistical Check
print("Raw Counts:")
print(df_train['target'].value_counts())

```

```
print("\nProportions:")
print(df_train['target'].value_counts(normalize=True))

# Visualization
plt.figure(figsize=(8, 5))
sns.countplot(x='target', data=df_train, palette='viridis')
plt.title('Class Distribution')
plt.xlabel('Class Label')
plt.ylabel('Number of Observations')
plt.show()
```

Raw Counts:

target

0 60988

1 39012

Name: count, dtype: int64

Proportions:

target

0 0.60988

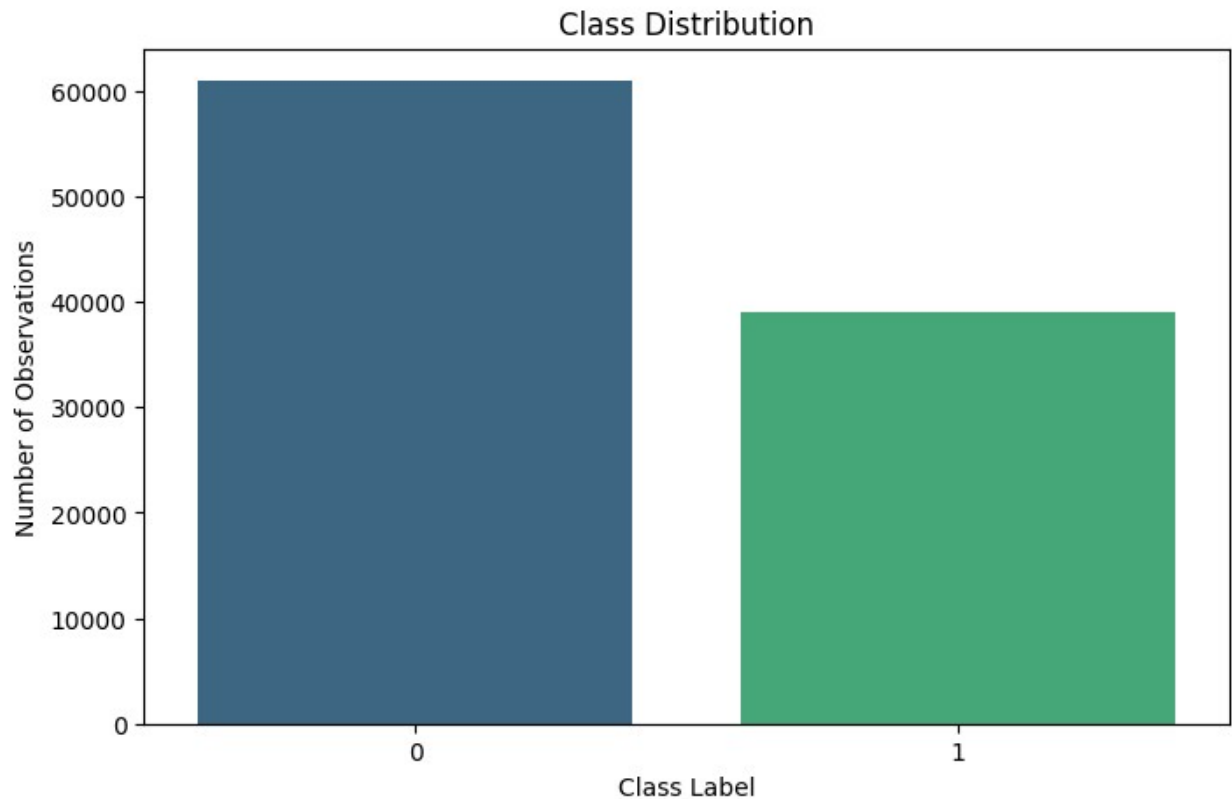
1 0.39012

Name: proportion, dtype: float64

C:\Users\nealb\AppData\Local\Temp\ipykernel_27340\2371519322.py:13:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='target', data=df_train, palette='viridis')
```



```
# import seaborn as sns
# import matplotlib.pyplot as plt

# # Statistical Check
# print("Raw Counts:")
# print(df_test['target'].value_counts())

# print("\nProportions:")
# print(df_test['target'].value_counts(normalize=True))

# # Visualization
# plt.figure(figsize=(8, 5))
# sns.countplot(x='target', data=df_test, palette='viridis')
# plt.title('Class Distribution')
# plt.xlabel('Class Label')
# plt.ylabel('Number of Observations')
# plt.show()
```

Class imbalance is observed in the dataset for both train and test data.

```
print(df_train.isna().sum())
# print(df_test.isna().sum())
```

person_id	0
age	0

gender	0
occupation	0
bmi	0
country	0
sleep_duration_hrs	0
sleep_quality_score	0
rem_percentage	0
deep_sleep_percentage	0
sleep_latency_mins	0
wake_episodes_per_night	0
caffeine_mg_before_bed	0
alcohol_units_before_bed	0
screen_time_before_bed_mins	0
exercise_day	0
steps_that_day	0
nap_duration_mins	0
stress_score	0
work_hours_that_day	0
chronotype	0
mental_health_condition	0
heart_rate_resting_bpm	0
sleep_aid_used	0
shift_work	0
room_temperature_celsius	0
weekend_sleep_diff_hrs	0
season	0
day_type	0
cognitive_performance_score	0
sleep_disorder_risk	0
target	0

dtype: int64

```
df_train.dropna(inplace=True)
# df_test.dropna(inplace=True)
print(df_train.isna().sum())
# print(df_test.isna().sum())
```

person_id	0
age	0
gender	0
occupation	0
bmi	0
country	0
sleep_duration_hrs	0
sleep_quality_score	0
rem_percentage	0
deep_sleep_percentage	0
sleep_latency_mins	0
wake_episodes_per_night	0
caffeine_mg_before_bed	0

```

alcohol_units_before_bed      0
screen_time_before_bed_mins   0
exercise_day                   0
steps_that_day                 0
nap_duration_mins              0
stress_score                   0
work_hours_that_day            0
chronotype                     0
mental_health_condition        0
heart_rate_resting_bpm         0
sleep_aid_used                 0
shift_work                     0
room_temperature_celsius       0
weekend_sleep_diff_hrs         0
season                         0
day_type                       0
cognitive_performance_score    0
sleep_disorder_risk            0
target                         0
dtype: int64

total_duplicates = df_train.duplicated().sum()
print(f"Total duplicate rows: {total_duplicates}")

Total duplicate rows: 0

# total_duplicates = df_test.duplicated().sum()
# print(f"Total duplicate rows: {total_duplicates}")

```

No duplicate rows were identified in the dataset.

```

df_train.drop(columns=["person_id"], inplace=True)
# df_test.drop(columns=["ID"], inplace=True)

```

The ID column was removed due to unique identifiers not contributing as predictive features for the dataset.

Data Transformation

Nominal and Ordinal Scaling of non-numeric Features

```

def unique_vals(df, column):
    # Extract an array of unique values
    unique_categories = df[column].unique()
    print("Unique Values Array:")
    print(unique_categories)

unique_vals(df_train, "sleep_disorder_risk")

```

```

Unique Values Array:
['Healthy' 'Severe' 'Mild' 'Moderate']

# unique_vals(df_train, "Spending_Score")

import pandas as pd
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder

# Separate features (X) and target (y) prior to transformations
X = df_train.drop('target', axis=1)
y = df_train['target']

# Define column classifications without overlaps
nominal_cols = [
    'gender',
    'occupation',
    'country',
    'chronotype',
    'mental_health_condition',
    'season',
    'day_type',
]

ordinal_cols = [
    'sleep_disorder_risk'
]

# Define the explicit order for ordinal categories
# These arrays must match the unique values in your actual dataset
# graduated_categories = ['No', 'Yes']
sleep_disorder_risk_categories = ['Healthy', 'Mild', 'Moderate',
                                   'Severe']

# Process nominal features using pandas get_dummies
X_train_encoded = pd.get_dummies(X, columns=nominal_cols)

# Process ordinal features using OrdinalEncoder
ordinal_encoder =
OrdinalEncoder(categories=[sleep_disorder_risk_categories])
X_train_encoded[ordinal_cols] =
ordinal_encoder.fit_transform(X_train_encoded[ordinal_cols])

# Process the target variable using LabelEncoder
# This encodes the target as integers for
sparse_categorical_crossentropy loss
label_encoder = LabelEncoder()
y_train_encoded = label_encoder.fit_transform(y)

# Verify the final data types and structure

```

```
print(X_train_encoded.info())
print(f"Target shape: {X_train_encoded.shape}")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 66 columns):
```

#	Column	Non-Null Count	Dtype
0	age	100000 non-null	int64
1	bmi	100000 non-null	float64
2	sleep_duration_hrs	100000 non-null	float64
3	sleep_quality_score	100000 non-null	float64
4	rem_percentage	100000 non-null	float64
5	deep_sleep_percentage	100000 non-null	float64
6	sleep_latency_mins	100000 non-null	int64
7	wake_episodes_per_night	100000 non-null	int64
8	caffeine_mg_before_bed	100000 non-null	int64
9	alcohol_units_before_bed	100000 non-null	float64
10	screen_time_before_bed_mins	100000 non-null	int64
11	exercise_day	100000 non-null	int64
12	steps_that_day	100000 non-null	int64
13	nap_duration_mins	100000 non-null	int64
14	stress_score	100000 non-null	float64
15	work_hours_that_day	100000 non-null	float64
16	heart_rate_resting_bpm	100000 non-null	int64
17	sleep_aid_used	100000 non-null	int64
18	shift_work	100000 non-null	int64
19	room_temperature_celsius	100000 non-null	float64
20	weekend_sleep_diff_hrs	100000 non-null	float64
21	cognitive_performance_score	100000 non-null	float64
22	sleep_disorder_risk	100000 non-null	float64
23	gender_Female	100000 non-null	bool
24	gender_Male	100000 non-null	bool
25	gender_Other	100000 non-null	bool
26	occupation_Doctor	100000 non-null	bool
27	occupation_Driver	100000 non-null	bool
28	occupation_Freelancer	100000 non-null	bool
29	occupation_Homemaker	100000 non-null	bool
30	occupation_Lawyer	100000 non-null	bool
31	occupation_Manager	100000 non-null	bool
32	occupation_Nurse	100000 non-null	bool
33	occupation_Retired	100000 non-null	bool
34	occupation_Sales	100000 non-null	bool
35	occupation_Software Engineer	100000 non-null	bool
36	occupation_Student	100000 non-null	bool
37	occupation_Teacher	100000 non-null	bool
38	country_Australia	100000 non-null	bool
39	country_Brazil	100000 non-null	bool
40	country_Canada	100000 non-null	bool
41	country_France	100000 non-null	bool

```

42 country_Germany 100000 non-null bool
43 country_India 100000 non-null bool
44 country_Italy 100000 non-null bool
45 country_Japan 100000 non-null bool
46 country_Mexico 100000 non-null bool
47 country_Netherlands 100000 non-null bool
48 country_South Korea 100000 non-null bool
49 country_Spain 100000 non-null bool
50 country_Sweden 100000 non-null bool
51 country_UK 100000 non-null bool
52 country_USA 100000 non-null bool
53 chronotype_Evening 100000 non-null bool
54 chronotype_Morning 100000 non-null bool
55 chronotype_Neutral 100000 non-null bool
56 mental_health_condition_Anxiety 100000 non-null bool
57 mental_health_condition_Both 100000 non-null bool
58 mental_health_condition_Depression 100000 non-null bool
59 mental_health_condition_Healthy 100000 non-null bool
60 season_Autumn 100000 non-null bool
61 season_Spring 100000 non-null bool
62 season_Summer 100000 non-null bool
63 season_Winter 100000 non-null bool
64 day_type_Weekday 100000 non-null bool
65 day_type_Weekend 100000 non-null bool
dtypes: bool(43), float64(12), int64(11)
memory usage: 21.6 MB
None
Target shape: (100000, 66)

y_train_encoded
array([0, 1, 0, ..., 1, 0, 0], dtype=int64)

# # Separate features (X) and target (y) prior to transformations
# X = df_test.drop('target', axis=1)
# y = df_test['target']

# # Process nominal features using pandas get_dummies
# X_test_encoded = pd.get_dummies(X, columns=nominal_cols)

# # Process ordinal features using OrdinalEncoder
# # ordinal_encoder = OrdinalEncoder(categories=[graduated_categories,
spending_categories])
# X_test_encoded[ordinal_cols] =
ordinal_encoder.fit_transform(X_test_encoded[ordinal_cols])

# # Process the target variable using LabelEncoder
# # This encodes the target as integers for
sparse_categorical_crossentropy loss
# # label_encoder = LabelEncoder()

```

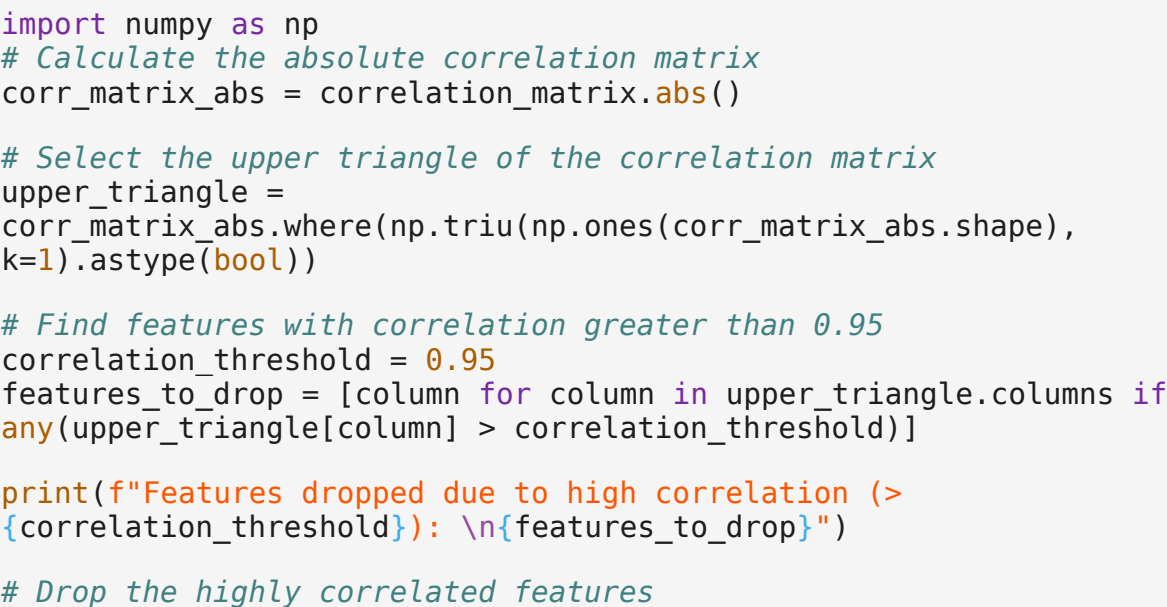
```
# y_test_encoded = label_encoder.fit_transform(y)

# # Verify the final data types and structure
# print(X_test_encoded.info())
# print(f"Target shape: {X_test_encoded.shape}")
```

The columns of the dataset were transformed depending on their nominal or ordinal nature. Among the data, only Graduated and Spending_Score were treated as ordinal data, while the rest of dtype "Object" were transform using Ordinal encoder of scikit-learn.

Exploring Feature Correlation

```
# Correlation matrix to assess feature redundancy
plt.figure(figsize=(32,24))
correlation_matrix = X_train_encoded.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',
fmt=".2f")
plt.title('Feature Correlation Matrix')
plt.show()
```

```
X_train_encoded = X_train_encoded.drop(columns=features_to_drop)
# X_test_encoded = X_test_encoded.drop(columns=features_to_drop)

Features dropped due to high correlation (> 0.95):
['gender_Male', 'day_type_Weekend']
```

Statistical Check

```
import pandas as pd

# Select only numerical features to avoid calculating stats on boolean
columns
num_cols = X_train_encoded.select_dtypes(include=['float64'])

# Generate standard descriptive statistics
descriptive_stats = num_cols.describe()

# Calculate and append skewness and kurtosis
descriptive_stats.loc['skewness'] = num_cols.skew()
descriptive_stats.loc['kurtosis'] = num_cols.kurtosis()

# Display the final summary table
print(descriptive_stats)
```

	bmi	sleep_duration_hrs	sleep_quality_score \
count	100000.000000	100000.000000	100000.000000
mean	26.289673	6.423986	4.871144
std	4.479578	1.274627	1.506517
min	16.000000	3.000000	1.000000
25%	23.200000	5.530000	3.800000
50%	26.300000	6.360000	4.900000
75%	29.300000	7.270000	6.000000
max	45.000000	10.500000	10.000000
skewness	0.071914	0.216783	-0.091709
kurtosis	-0.180689	-0.078708	-0.419022

	rem_percentage	deep_sleep_percentage
alcohol_units_before_bed		\
count	100000.000000	100000.000000
100000.000000		
mean	20.243968	20.253375
0.596800		
std	3.411354	4.251096
1.059161		
min	10.000000	5.000000
0.000000		
25%	18.000000	17.400000
0.000000		
50%	20.300000	20.300000
0.000000		

75%	22.600000	23.200000
1.000000		
max	30.000000	30.000000
6.000000		
skewness	-0.113301	-0.108602
1.927083		
kurtosis	-0.061205	-0.174163
3.445118		

	stress_score	work_hours_that_day	room_temperature_celsius
\			
count	100000.000000	100000.000000	100000.000000
mean	5.733285	7.134059	20.538093
std	1.619194	3.482878	2.897565
min	1.000000	0.000000	15.000000
25%	4.800000	4.700000	18.500000
50%	5.800000	7.400000	20.500000
75%	6.800000	9.700000	22.500000
max	10.000000	18.000000	28.000000
skewness	-0.363788	-0.161502	0.128792
kurtosis	0.251535	-0.527978	-0.438709

	weekend_sleep_diff_hrs	cognitive_performance_score	\
count	100000.000000	100000.000000	
mean	1.200634	59.227295	
std	0.786667	22.249665	
min	-1.000000	0.000000	
25%	0.670000	44.200000	
50%	1.200000	60.400000	
75%	1.740000	75.800000	
max	3.000000	100.000000	
skewness	-0.052748	-0.289882	
kurtosis	-0.256679	-0.412655	

	sleep_disorder_risk
count	100000.000000
mean	0.622750
std	0.803044
min	0.000000
25%	0.000000

50%	0.000000
75%	1.000000
max	3.000000
skewness	1.255640
kurtosis	1.052910

Checking continuous features for skewness

```
print(X_train_encoded.head())
```

	age	bmi	sleep_duration_hrs	sleep_quality_score	rem_percentage
0	29	25.7	6.19	6.6	22.5
1	55	22.0	8.32	6.9	26.9
2	42	25.0	3.74	1.0	20.2
3	37	29.5	6.79	6.4	17.7
4	23	23.6	5.02	3.2	23.3

	deep_sleep_percentage	sleep_latency_mins	wake_episodes_per_night
0	19.3	16	3
1	14.9	17	4
2	16.2	26	4
3	17.7	13	4
4	18.3	30	5

	caffeine_mg_before_bed	alcohol_units_before_bed	...
chronotype_Neutral			
0	0	0.0	...
False			
1	0	0.0	...
True			
2	0	2.0	...
True			
3	0	1.0	...
False			
4	40	0.0	...
True			

	mental_health_condition_Anxiety	mental_health_condition_Both	...
0	False	False	...

1	False	False
2	False	True
3	False	False
4	False	False

	mental_health_condition_Depression	mental_health_condition_Healthy
\		
0	False	True
1	False	True
2	False	False
3	False	True
4	False	True

	season_Autumn	season_Spring	season_Summer	season_Winter	\
0	True	False	False	False	
1	False	False	False	True	
2	False	True	False	False	
3	False	False	True	False	
4	False	False	True	False	

	day_type_Weekday
0	True
1	False
2	False
3	False
4	True

[5 rows x 64 columns]

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
# Identify numeric columns dynamically
# A threshold for unique values helps distinguish continuous variables
```

```
# from discrete features that may also be represented by numeric types.
```

```
unique_threshold = 2
continuous_cols = [
    col for col in X_train_encoded.select_dtypes(include=[np.number,
np.float64, np.int64]).columns
    if X_train_encoded[col].nunique() > unique_threshold
]
```

```
# Assess Skewness on Continuous Variables Only
```

```

continuous_skewness =
X_train_encoded[continuous_cols].skew().sort_values(ascending=False)

print("Continuous Feature Skewness:")
print(continuous_skewness)

# Calculate grid size dynamically to accommodate the identified
columns
num_plots = len(continuous_cols)
ncols = 3
nrows = (num_plots + ncols - 1) // ncols

# Create a grid of subplots for the distributions
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(15, 6 *
nrows))
fig.subplots_adjust(hspace=0.4, wspace=0.3)

# Ensure axes is an iterable array even if only one plot exists
if num_plots > 1:
    axes = axes.ravel()
else:
    axes = np.array([axes])

# Plot a histogram for each continuous variable
for i, col in enumerate(continuous_cols):
    # Drop NA values to prevent matplotlib errors during rendering
    axes[i].hist(X_train_encoded[col].dropna(), bins=30,
edgecolor='black', alpha=0.7)
    axes[i].set_title(f'Distribution of {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Frequency')

# Hide any unused subplots in the grid
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

plt.tight_layout()

# Save the figure or display it depending on your environment
plt.savefig("continuous_distributions.png")
plt.show()

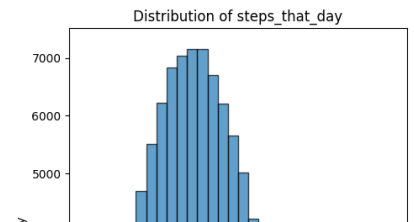
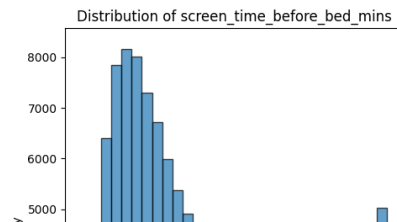
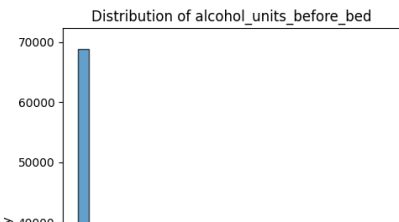
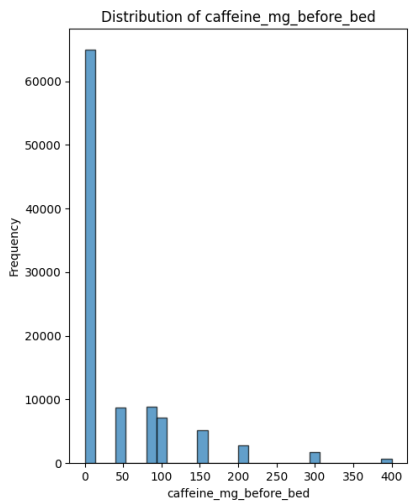
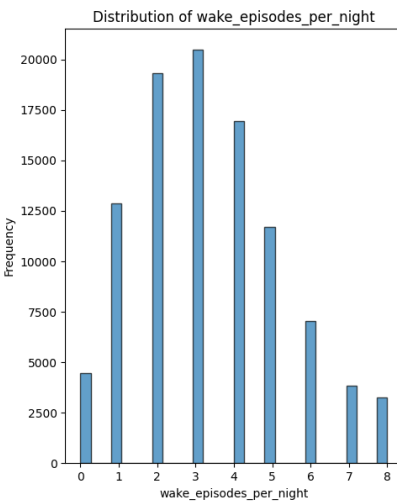
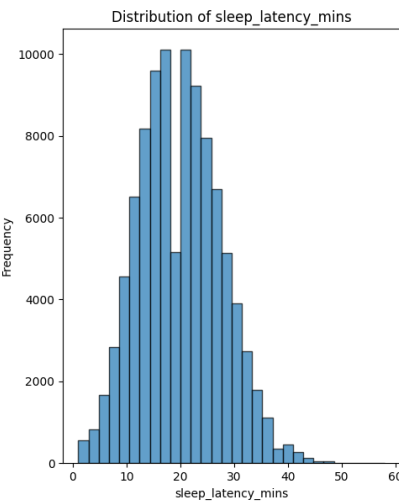
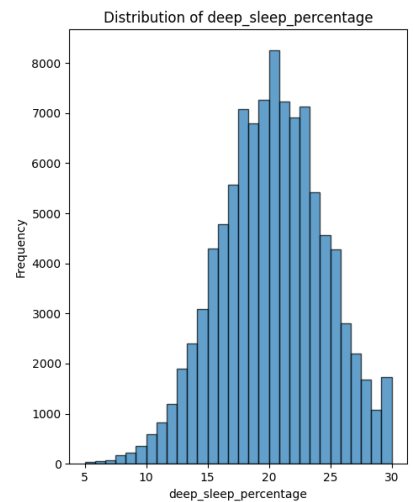
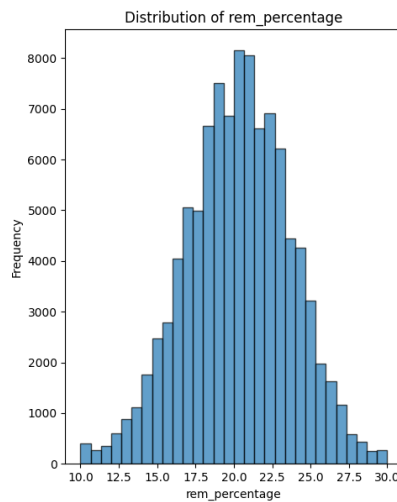
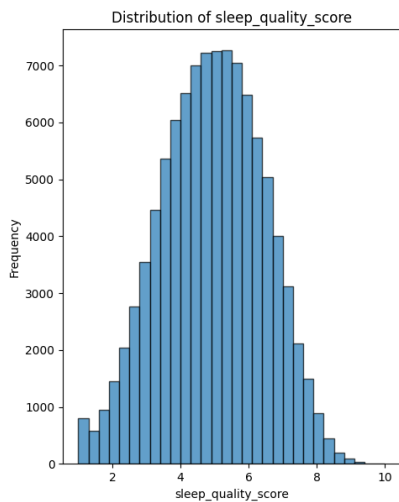
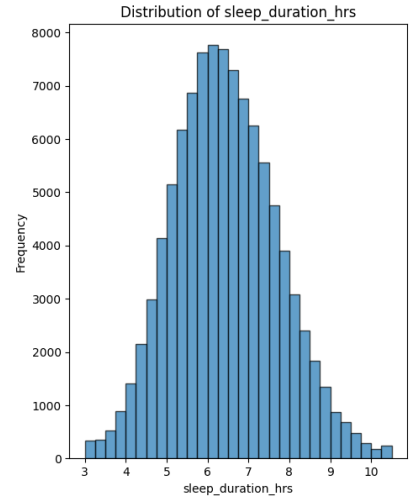
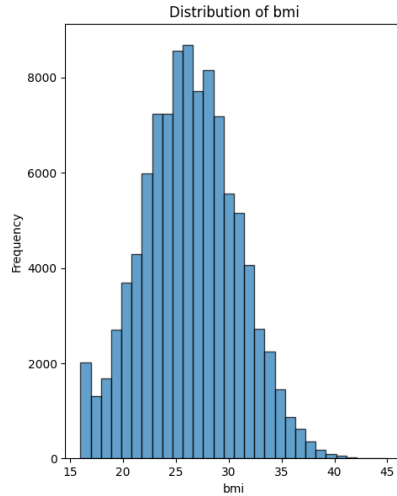
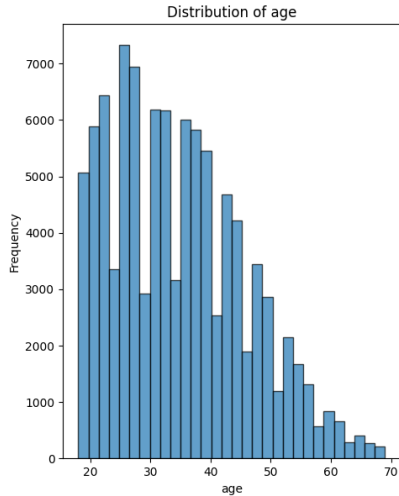
```

```

Continuous Feature Skewness:
caffeine_mg_before_bed      2.384080
alcohol_units_before_bed    1.927083
sleep_disorder_risk         1.255640
nap_duration_mins           1.205569
screen_time_before_bed_mins 1.152855
age                         0.549902
wake_episodes_per_night     0.454924

```

sleep_latency_mins	0.242292
sleep_duration_hrs	0.216783
steps_that_day	0.135000
room_temperature_celsius	0.128792
bmi	0.071914
heart_rate_resting_bpm	0.005647
weekend_sleep_diff_hrs	-0.052748
sleep_quality_score	-0.091709
deep_sleep_percentage	-0.108602
rem_percentage	-0.113301
work_hours_that_day	-0.161502
cognitive_performance_score	-0.289882
stress_score	-0.363788
dtype:	float64



```

from sklearn.model_selection import train_test_split

X = X_train_encoded
y = y_train_encoded

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train,
test_size=0.2, random_state=42)

print(f"Training set size: {X_train.shape[0]} samples")
print(f"Validation set size: {X_val.shape[0]} samples")
print(f"Test set size: {X_test.shape[0]} samples")

Training set size: 64000 samples
Validation set size: 16000 samples
Test set size: 20000 samples

import pandas as pd
import numpy as np
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import PowerTransformer, MinMaxScaler,
RobustScaler, StandardScaler

# Update columns to address the observed right skews
skewed_cols = ['age']

outlier_heavy_cols = ['nap_duration_mins',
'screen_time_before_bed_mins']

standard_cols = ['bmi', 'sleep_quality_score', 'sleep_quality_score',
'rem_percentage', 'deep_sleep_percentage', 'sleep_latency_mins',
'steps_that_day', 'stress_score', 'work_hours_that_day',
'heart_rate_resting_bpm', 'room_temperature_celsius',
'weekend_sleep_diff_hrs', 'cognitive_performance_score']

minmax_cols = ['caffeine_mg_before_bed', 'alcohol_units_before_bed' ,
'sleep_disorder_risk', 'wake_episodes_per_night']

preprocessor = ColumnTransformer(
    transformers=[
        ('power', PowerTransformer(method='yeo-johnson'),
skewed_cols),
        ('robust', RobustScaler(), outlier_heavy_cols),
        ('standard', StandardScaler(), standard_cols),
        ('minmax', StandardScaler(), minmax_cols)
    ],

```

```

        remainder='passthrough'
    )

    # Fit and transform the datasets
    X_train_scaled = preprocessor.fit_transform(X_train)
    X_test_scaled = preprocessor.transform(X_test)

    # Explicitly cast the resulting object arrays to float32 for TensorFlow
    X_train_scaled = np.asarray(X_train_scaled).astype(np.float32)
    X_test_scaled = np.asarray(X_test_scaled).astype(np.float32)

    # Verify the final data type
    print(f"X_train_scaled dtype: {X_train_scaled.dtype}")

X_train_scaled dtype: float32

```

Age was handled as a skewed column and transformed using yeo-johnson, while spending score and family size was transformed using min max. The rest of the columns just pass through as they are already binari encoded (one-hot)

Exploring and Handling Class Imblance using SMOTE

```

import pandas as pd
from imblearn.over_sampling import SMOTE

# Initialize SMOTE
smote = SMOTE(random_state=42)

# Apply SMOTE strictly to the training data
X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
y_train)

# Verify the class distribution before and after SMOTE
print("Original Training Target Distribution:")
print(pd.Series(y_train).value_counts())

print("\n" + "="*40 + "\n")

print("SMOTE Training Target Distribution:")
print(pd.Series(y_train_resampled).value_counts())

Original Training Target Distribution:
0    39135
1     24865
Name: count, dtype: int64

=====

SMOTE Training Target Distribution:

```



```
1    39135
0    39135
Name: count, dtype: int64
```

The training data class imbalance was handled with a number of 1387 datapoints now for each class.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Create a DataFrame and explicitly name the column
df_resampled = pd.DataFrame(y_train_resampled, columns=['target'])

# Visualization
plt.figure(figsize=(8, 5))

# Use the string column name for both x and hue to safely apply the
palette
sns.countplot(data=df_resampled, x='target', hue='target',
palette='viridis', legend=False)

plt.title('Class Distribution After SMOTE')
plt.xlabel('Class Label')
plt.ylabel('Number of Observations')
plt.show()
```



```
y_train_resampled[:10]  
array([1, 0, 0, 1, 0, 1, 1, 1, 0, 0], dtype=int64)
```

Build a Baseline ANN Model

Creating baseline ANN model based on

```
import tensorflow as tf  
  
# Verify GPU availability  
gpus = tf.config.list_physical_devices('GPU')  
if gpus:  
    try:  
        # Enable memory growth so TF only takes what it needs  
        for gpu in gpus:  
            tf.config.experimental.set_memory_growth(gpu, True)  
        print(f"GPU detected: {gpus}")  
    except RuntimeError as e:  
        print(e)  
else:  
    print("No GPU detected. Check your CUDA/cuDNN installation.")
```

```

GPU detected: [PhysicalDevice(name='/physical_device:GPU:0',
device_type='GPU')]

import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))
[PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]

import tensorflow as tf

# Check if TensorFlow is linked to CUDA
print("Built with CUDA:", tf.test.is_built_with_cuda())

# Check GPU details and recognized CUDA version
gpu_devices = tf.config.list_physical_devices('GPU')
if gpu_devices:
    # This specifically checks the version linked to the binary
    from tensorflow.python.platform import build_info as tf_build_info
    print("TF CUDA Version:",
    tf_build_info.build_info['cuda_version'])
    print("TF cuDNN Version:",
    tf_build_info.build_info['cudnn_version'])
else:
    print("No GPU detected by TensorFlow.")

Built with CUDA: True
TF CUDA Version: 64_112
TF cuDNN Version: 64_8

# Convert features to float32 for model weights
X_train_resampled = np.array(X_train_resampled).astype('float32')
X_val = np.array(X_val).astype('float32')
X_test = np.array(X_test).astype('float32')

# Convert labels to int32
y_train_resampled = np.array(y_train_resampled).astype('int32')
y_val = np.array(y_val).astype('int32')
y_test = np.array(y_test).astype('int32')

```

Baseline Model Training

```

import numpy as np
from keras.models import Model
from keras.layers import Input, Dense

# Determine dynamic shapes directly from the resampled data
input_dim = X_train_resampled.shape[1]
num_classes = len(np.unique(y_train_resampled))

# Defining the ANN
input_layer = Input(shape=(input_dim,), name='input_1')

```

```
# Dense Layers
x = Dense(32, activation='relu', name='dense_1')(input_layer)
x = Dense(32, activation='relu', name='dense_2')(x)
```

```
# Output layer configured for multi-class classification
output_layer = Dense(num_classes, activation='softmax',
name='output_1')(x)
```

```
# Compiling the model
model = Model(inputs=input_layer, outputs=output_layer,
name='baseline_model')
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
# Displaying the model parameters
model.summary()
```

```
Model: "baseline_model"
```

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 64)]	0
dense_1 (Dense)	(None, 32)	2080
dense_2 (Dense)	(None, 32)	1056
output_1 (Dense)	(None, 2)	66

```
====
Total params: 3,202
Trainable params: 3,202
Non-trainable params: 0
=====
```

```
## Training/Fitting the model
```

```
baseline_history = model.fit(X_train_resampled, y_train_resampled,
validation_data=(X_val, y_val), epochs=32)
```

```
Epoch 1/32
2446/2446 [=====] - 11s 3ms/step - loss:
4.1307 - accuracy: 0.6298 - val_loss: 2.0825 - val_accuracy: 0.6670
Epoch 2/32
2446/2446 [=====] - 8s 3ms/step - loss:
2.5403 - accuracy: 0.6672 - val_loss: 2.2719 - val_accuracy: 0.6655
Epoch 3/32
2446/2446 [=====] - 9s 4ms/step - loss:
2.3650 - accuracy: 0.6779 - val_loss: 3.0031 - val_accuracy: 0.6412
Epoch 4/32
2446/2446 [=====] - 9s 3ms/step - loss:
```

1.7543 - accuracy: 0.6944 - val_loss: 5.1300 - val_accuracy: 0.6197
Epoch 5/32
2446/2446 [=====] - 8s 3ms/step - loss:
1.6265 - accuracy: 0.6988 - val_loss: 3.2268 - val_accuracy: 0.5300
Epoch 6/32
2446/2446 [=====] - 8s 3ms/step - loss:
1.5200 - accuracy: 0.7008 - val_loss: 0.7144 - val_accuracy: 0.7327
Epoch 7/32
2446/2446 [=====] - 8s 3ms/step - loss:
1.2626 - accuracy: 0.7062 - val_loss: 0.6496 - val_accuracy: 0.7297
Epoch 8/32
2446/2446 [=====] - 8s 3ms/step - loss:
1.0470 - accuracy: 0.7114 - val_loss: 0.9314 - val_accuracy: 0.6681
Epoch 9/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.8473 - accuracy: 0.7166 - val_loss: 0.5690 - val_accuracy: 0.7375
Epoch 10/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.7779 - accuracy: 0.7230 - val_loss: 1.4354 - val_accuracy: 0.6312
Epoch 11/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.6885 - accuracy: 0.7243 - val_loss: 1.3758 - val_accuracy: 0.5480
Epoch 12/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.6338 - accuracy: 0.7289 - val_loss: 0.6206 - val_accuracy: 0.7023
Epoch 13/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.6080 - accuracy: 0.7304 - val_loss: 0.5202 - val_accuracy: 0.7377
Epoch 14/32
2446/2446 [=====] - 9s 3ms/step - loss:
0.5512 - accuracy: 0.7399 - val_loss: 0.6019 - val_accuracy: 0.6926
Epoch 15/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.5332 - accuracy: 0.7431 - val_loss: 0.5182 - val_accuracy: 0.7356
Epoch 16/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.5191 - accuracy: 0.7450 - val_loss: 0.5262 - val_accuracy: 0.7321
Epoch 17/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.5009 - accuracy: 0.7524 - val_loss: 0.5249 - val_accuracy: 0.7309
Epoch 18/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.4974 - accuracy: 0.7533 - val_loss: 0.5116 - val_accuracy: 0.7369
Epoch 19/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.4895 - accuracy: 0.7572 - val_loss: 0.5400 - val_accuracy: 0.7272
Epoch 20/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4862 - accuracy: 0.7576 - val_loss: 0.5201 - val_accuracy: 0.7296

```

Epoch 21/32
2446/2446 [=====] - 9s 3ms/step - loss:
0.4750 - accuracy: 0.7639 - val_loss: 0.5921 - val_accuracy: 0.6929
Epoch 22/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.4713 - accuracy: 0.7649 - val_loss: 0.6299 - val_accuracy: 0.6747
Epoch 23/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.4722 - accuracy: 0.7639 - val_loss: 0.5298 - val_accuracy: 0.7211
Epoch 24/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4719 - accuracy: 0.7646 - val_loss: 0.5512 - val_accuracy: 0.7182
Epoch 25/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4643 - accuracy: 0.7678 - val_loss: 0.5523 - val_accuracy: 0.7084
Epoch 26/32
2446/2446 [=====] - 8s 3ms/step - loss:
0.4657 - accuracy: 0.7645 - val_loss: 0.5574 - val_accuracy: 0.6997
Epoch 27/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4619 - accuracy: 0.7672 - val_loss: 0.5153 - val_accuracy: 0.7296
Epoch 28/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4596 - accuracy: 0.7687 - val_loss: 0.5377 - val_accuracy: 0.7237
Epoch 29/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4572 - accuracy: 0.7684 - val_loss: 0.5205 - val_accuracy: 0.7251
Epoch 30/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4532 - accuracy: 0.7706 - val_loss: 0.5129 - val_accuracy: 0.7331
Epoch 31/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4524 - accuracy: 0.7700 - val_loss: 0.5160 - val_accuracy: 0.7281
Epoch 32/32
2446/2446 [=====] - 9s 4ms/step - loss:
0.4527 - accuracy: 0.7701 - val_loss: 0.5114 - val_accuracy: 0.7352

```

```
# @title Function for Multi Class Evaluation
```

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score,
                             f1_score, confusion_matrix, roc_curve,
                             auc)
from sklearn.preprocessing import label_binarize

def plot_training_history(history_obj):
    """
    Plots the training and validation loss and accuracy per epoch.

```

```

Accepts either a Keras History object or a history dictionary.
"""
# Extract the dictionary if a Keras History object is passed
history_dict = history_obj.history if hasattr(history_obj,
'history') else history_obj

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss Plot
axes[0].plot(history_dict['loss'], label='Train Loss',
color='blue', linewidth=2)
if 'val_loss' in history_dict:
    axes[0].plot(history_dict['val_loss'], label='Validation
Loss', color='orange', linewidth=2)
axes[0].set_title('Model Loss per Epoch')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend(loc='upper right')
axes[0].grid(True, linestyle='--', alpha=0.6)

# Accuracy Plot
axes[1].plot(history_dict['accuracy'], label='Train Accuracy',
color='blue', linewidth=2)
if 'val_accuracy' in history_dict:
    axes[1].plot(history_dict['val_accuracy'], label='Validation
Accuracy', color='orange', linewidth=2)
axes[1].set_title('Model Accuracy per Epoch')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend(loc='lower right')
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import (accuracy_score, precision_score,
recall_score,
                                f1_score, confusion_matrix, roc_curve,
auc)
from sklearn.preprocessing import label_binarize

def evaluate_multiclass_model(model, X_set, y_set, n_classes,
scikeras=False):
    """
    Evaluates a trained binary or multi-class ANN model and prints

```

```

performance metrics.
Generates a Confusion Matrix and a relevant ROC Curve.
Returns a dictionary containing the numeric evaluation metrics.
"""
# Inside your evaluate_multiclass_model function:
if scikeras:
    y_prob = model.predict_proba(X_set)
else:
    # Native Keras models return probabilities via .predict()
    y_prob = model.predict(X_set)

# Determine predicted classes based on output shape
if y_prob.ndim == 1 or y_prob.shape[1] == 1:
    # Binary sigmoid case: shape is (N,) or (N, 1)
    y_prob_clean = y_prob.flatten()
    y_pred = (y_prob_clean >= 0.5).astype(int)
else:
    # Multi-class softmax case: shape is (N, n_classes)
    y_pred = np.argmax(y_prob, axis=1)
    y_prob_clean = y_prob

# Calculate metrics using macro average for consistent support
accuracy = accuracy_score(y_set, y_pred)
precision = precision_score(y_set, y_pred, average='macro',
zero_division=0)
recall = recall_score(y_set, y_pred, average='macro',
zero_division=0)
f1 = f1_score(y_set, y_pred, average='macro', zero_division=0)

print(f"Accuracy: {accuracy:.4f}")
print(f"Precision (Macro): {precision:.4f}")
print(f"Recall (Macro): {recall:.4f}")
print(f"F1-score (Macro): {f1:.4f}\n")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Confusion Matrix
cm = confusion_matrix(y_set, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[0])
axes[0].set_title('Confusion Matrix')
axes[0].set_xlabel('Predicted Label')
axes[0].set_ylabel('True Label')

# ROC Curve and AUC Logic
if n_classes == 2:
    # Binary case: label_binarize returns (N, 1), so we use y_set
    directly
    # Use probabilities of the positive class
    pos_prob = y_prob_clean[:, 1] if y_prob_clean.ndim > 1 else
y_prob_clean

```



```

    fpr, tpr, _ = roc_curve(y_set, pos_prob)
    roc_auc = auc(fpr, tpr)
    axes[1].plot(fpr, tpr, lw=2, label=f'Binary ROC (AUC =
{roc_auc:.2f})')
else:
    # Multi-class One-vs-Rest approach
    y_set_binarized = label_binarize(y_set,
classes=range(n_classes))
    for i in range(n_classes):
        fpr, tpr, _ = roc_curve(y_set_binarized[:, i],
y_prob_clean[:, i])
        roc_auc = auc(fpr, tpr)
        axes[1].plot(fpr, tpr, lw=2, label=f'Class {i} (AUC =
{roc_auc:.2f})')

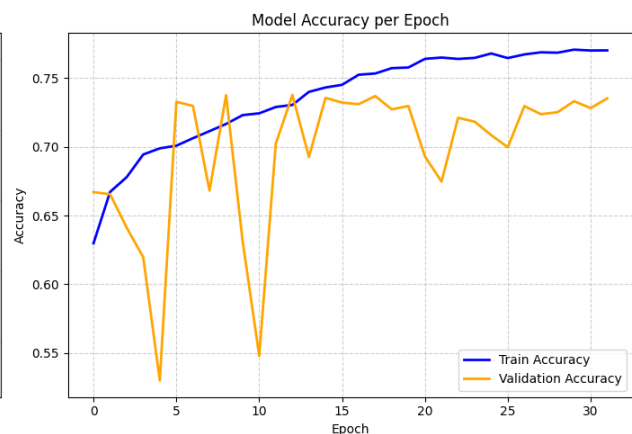
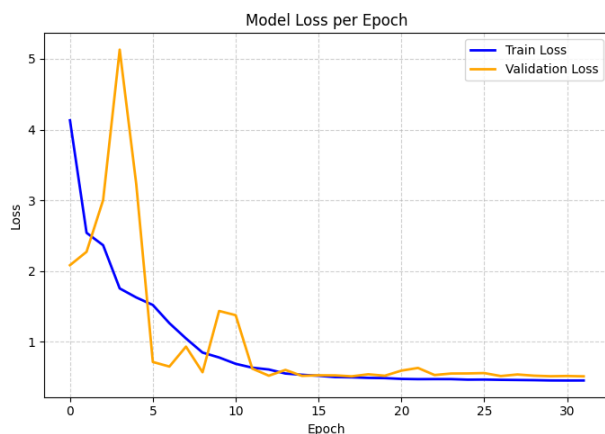
axes[1].plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
axes[1].set_xlim([0.0, 1.0])
axes[1].set_ylim([0.0, 1.05])
axes[1].set_xlabel('False Positive Rate')
axes[1].set_ylabel('True Positive Rate')
axes[1].set_title('Receiver Operating Characteristic')
axes[1].legend(loc="lower right", fontsize='small')

plt.tight_layout()
plt.show()

# Return the metrics in a pandas DataFrame
df = pd.DataFrame([
    'Accuracy': accuracy,
    'Precision (Macro)': precision,
    'Recall (Macro)': recall,
    'F1-score (Macro)': f1
])
return df

plot_training_history(baseline_history)

```



```
baseline_metrics = evaluate_multiclass_model(model, X_test, y_test, 2)
```

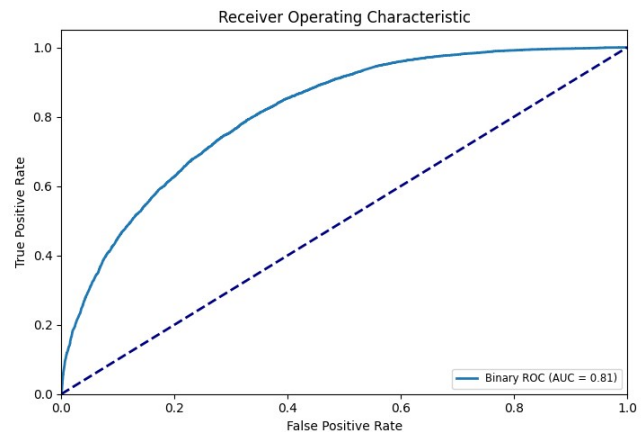
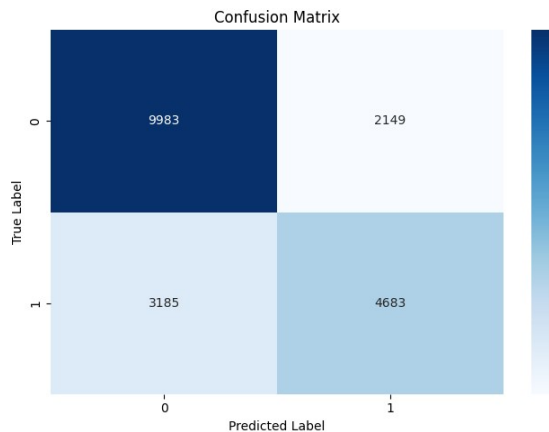
```
625/625 [=====] - 1s 1ms/step
```

```
Accuracy: 0.7333
```

```
Precision (Macro): 0.7218
```

```
Recall (Macro): 0.7090
```

```
F1-score (Macro): 0.7132
```



Saving Baseline Results

```
# Save Training History to CSV
```

```
history_df = pd.DataFrame(baseline_history.history)
```

```
history_df.to_csv('baseline_training_history.csv',  
index_label='epoch')
```

```
# Save Evaluation Metrics to CSV
```

```
baseline_metrics.to_csv('baseline_model_metrics.csv', index=False)
```

Optimized Model Training

```
# import numpy as np
```

```
# import pandas as pd
```

```
# import joblib
```

```
# import contextlib
```

```
# from tqdm import tqdm
```

```
# from sklearn.model_selection import RandomizedSearchCV
```

```
# from scipy.stats import loguniform, randint
```

```
# from scikeras.wrappers import KerasClassifier
```

```
# from tensorflow.keras.models import Model
```

```
# from tensorflow.keras.optimizers import Adam, SGD, RMSprop, Nadam,  
AdamW
```

```
# from tensorflow.keras.losses import SparseCategoricalCrossentropy
```

```
# from tensorflow.keras.layers import Input, Dense, Dropout,  
BatchNormalization, LeakyReLU
```

```
# from tensorflow.keras.callbacks import EarlyStopping
```

```

# # Context manager to patch joblib and report progress to tqdm
# @contextlib.contextmanager
# def tqdm_joblib(tqdm_object):
#     class
TqdmBatchCompletionCallback(joblib.parallel.BatchCompletionCallBack):
#         def __call__(self, *args, **kwargs):
#             tqdm_object.update(n=self.batch_size)
#             return super().__call__(*args, **kwargs)

#     old_batch_callback = joblib.parallel.BatchCompletionCallBack
#     joblib.parallel.BatchCompletionCallBack =
TqdmBatchCompletionCallback
#     try:
#         yield tqdm_object
#     finally:
#         joblib.parallel.BatchCompletionCallBack = old_batch_callback

# def create_model(num_layers=1, units=32, activation_func='relu',
#                 optimizer_name='adam', learning_rate=0.001,
#                 n_classes=4):
#     input_layer = Input(shape=(X_train_resampled.shape[1],),
# name="input_1")
#     x = input_layer

#     for i in range(num_layers):
#         if activation_func == 'leaky_relu':
#             x = Dense(units, name=f"dense_{i+1}")(x)
#             x = LeakyReLU(alpha=0.01, name=f"leaky_relu_{i+1}")(x)
#         else:
#             x = Dense(units, activation=activation_func,
# name=f"dense_{i+1}")(x)

#         x = BatchNormalization(name=f"batch_norm_{i+1}")(x)
#         x = Dropout(0.2, name=f"dropout_{i+1}")(x)

#     output_layer = Dense(n_classes, activation='softmax',
# name="output_1")(x)
#     model = Model(inputs=input_layer, outputs=output_layer,
# name="optimized_classifier")

#     optimizers = {
#         'adam': Adam(learning_rate=learning_rate),
#         'sgd': SGD(learning_rate=learning_rate),
#         'rmsprop': RMSprop(learning_rate=learning_rate),
#         'adamw': AdamW(learning_rate=learning_rate),
#         'nadam': Nadam(learning_rate=learning_rate)
#     }

#     opt = optimizers.get(optimizer_name,

```

```

Adam(learning_rate=learning_rate))

#     model.compile(
#         optimizer=opt,
#         loss=SparseCategoricalCrossentropy(),
#         metrics=['accuracy']
#     )
#     return model

# early_stopping = EarlyStopping(
#     monitor='val_loss',
#     patience=15,
#     restore_best_weights=True
# )

# keras_estimator = KerasClassifier(
#     model=create_model,
#     model__n_classes=4,
#     verbose=0,
#     callbacks=[early_stopping],
# )

# param_distributions = {
#     'model__num_layers': randint(1, 6),
#     'model__units': [32, 64, 128, 256, 512],
#     'model__activation_func': ['relu', 'tanh', 'leaky_relu', 'elu'],
#     'model__optimizer_name': ['adam', 'adamw', 'nadam'],
#     'model__learning_rate': loguniform(1e-4, 1e-2),
#     'batch_size': [32, 64, 128, 256, 512, 1024],
#     'epochs': [32, 64, 128, 256, 512]
# }

# n_iter = 50
# cv = 3
# random_search = RandomizedSearchCV(
#     estimator=keras_estimator,
#     param_distributions=param_distributions,
#     n_iter=n_iter,
#     cv=cv,
#     scoring='accuracy',
#     n_jobs=8,
#     random_state=42
# )

# print("Starting randomized search for classification...")
# # Calculate total fits: n_iter * cv
# with tqdm_joblib(tqdm(desc="RandomSearch", total=n_iter * cv)) as
progress_bar:
#     search_result = random_search.fit(
#         X_train_resampled,

```

```

#         y_train_resampled,
#         validation_data=(X_val, y_val)
#     )

# final_model = search_result.best_estimator_
# print(f"\nBest F1 Score: {search_result.best_score_:.4f}")
# print(f"Best Parameters: {search_result.best_params_}")

import os
import pandas as pd
import numpy as np
import tensorflow as tf
import tensorflow_addons as tfa
import keras_tuner as kt
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Dropout,
BatchNormalization, LeakyReLU
from tensorflow.keras.optimizers import Adam, RMSprop, Nadam
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau, CSVLogger

# 1. Hardware and Data Setup
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    for gpu in gpus:
        tf.config.experimental.set_memory_growth(gpu, True)

X_train_resampled = X_train_resampled.astype('float32')
y_train_resampled = y_train_resampled.astype('int32')
X_val = X_val.astype('float32')
y_val = y_val.astype('int32')
num_classes = len(np.unique(y_train_resampled))

class MyBayesianTuner(kt.BayesianOptimization):
    def run_trial(self, trial, x, y, validation_data, **kwargs):
        hp = trial.hyperparameters
        batch_size = hp.Choice('batch_size', [32, 64, 128, 256, 512,
1024])

        # Create a directory for this specific trial's logs
        trial_dir = os.path.join(self.directory, self.project_name,
f"trial_{trial.trial_id}")
        os.makedirs(trial_dir, exist_ok=True)

        # Define a CSVLogger for this trial
        history_file = os.path.join(trial_dir, "trial_history.csv")
        csv_logger = CSVLogger(history_file)

        # Create a clean copy of the callbacks list to prevent
        accumulation

```

```

trial_callbacks = kwargs.get('callbacks', []).copy()
trial_callbacks.append(csv_logger)
kwargs['callbacks'] = trial_callbacks

# Prepare datasets
train_ds = tf.data.Dataset.from_tensor_slices((x, y))
train_ds = (train_ds.shuffle(10000)
            .batch(batch_size)
            .cache()
            .prefetch(tf.data.AUTOTUNE))

val_x, val_y = validation_data
val_ds = (tf.data.Dataset.from_tensor_slices((val_x, val_y))
          .batch(batch_size)
          .cache()
          .prefetch(tf.data.AUTOTUNE))

# Run the trial
return super(MyBayesianTuner, self).run_trial(
    trial, train_ds, validation_data=val_ds, **kwargs
)

# 3. Enhanced Model Builder
def build_model(hp):
    input_layer = Input(shape=(X_train_resampled.shape[1],),
name="input_1")
    x = input_layer

    num_layers = hp.Int('num_layers', 1, 5)
    activation_func = hp.Choice('activation_func', ['relu',
'leaky_relu', 'elu'])
    dropout_rate = hp.Float('dropout_rate', 0.0, 0.4, step=0.1)
    batchnorm = hp.Boolean('batchnorm')

    for i in range(num_layers):
        units = hp.Choice(f'units_{i}', [32, 64, 128, 256, 512])

        if activation_func == 'leaky_relu':
            x = Dense(units, name=f"dense_{i+1}")(x)
            x = LeakyReLU(alpha=0.01, name=f"leaky_relu_{i+1}")(x)
        else:
            x = Dense(units, activation=activation_func,
name=f"dense_{i+1}")(x)

        if batchnorm:
            x = BatchNormalization(name=f"batch_norm_{i+1}")(x)
            x = Dropout(dropout_rate, name=f"dropout_{i+1}")(x)

    output_layer = Dense(num_classes, activation='softmax',
name="output_1")(x)

```

```

    model = Model(inputs=input_layer, outputs=output_layer)

    learning_rate = hp.Float('learning_rate', 1e-4, 1e-2,
sampling='log')
    optimizer_name = hp.Choice('optimizer_name', ['adam', 'rmsprop',
'adamw', 'nadam'])

    if optimizer_name == 'adam':
        opt = Adam(learning_rate=learning_rate)
    elif optimizer_name == 'rmsprop':
        opt = RMSprop(learning_rate=learning_rate)
    elif optimizer_name == 'adamw':
        wd = hp.Float('weight_decay', 1e-5, 1e-3, sampling='log')
        opt = tf.optimizers.AdamW(weight_decay=wd,
learning_rate=learning_rate)
    elif optimizer_name == 'nadam':
        opt = Nadam(learning_rate=learning_rate)

    model.compile(
        optimizer=opt,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model

# 4. Search Execution
tuner = MyBayesianTuner(
    build_model,
    objective='val_loss',
    max_trials=50,
    directory='bayesian_tuning_v4',
    project_name='customer_seg_final',
    overwrite=False
)

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=15,
    restore_best_weights=True
)

reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.2,
    patience=5,
    min_lr=1e-8
)

print("Starting final Bayesian Optimization...")
tuner.search(

```

```

x=X_train_resampled,
y=y_train_resampled,
validation_data=(X_val, y_val),
epochs=512,
callbacks=[early_stopping, reduce_lr]
)

```

Reloading Tuner from bayesian_tuning_v4\customer_seg_final\tuner0.json
Starting final Bayesian Optimization...

```

import os
import pandas as pd
import matplotlib.pyplot as plt

# 1. Retrieve the top 5 trials based on the objective (val_loss)
top_n = 5
top_trials = tuner.oracle.get_best_trials(num_trials=top_n)

# 2. Setup the visualization grid (2 rows: Loss and Accuracy | 5
columns: Trials)
fig, axes = plt.subplots(2, top_n, figsize=(25, 10), sharex='col')
plt.subplots_adjust(hspace=0.3, wspace=0.3)

for i, trial in enumerate(top_trials):
    trial_id = trial.trial_id
    history_path = os.path.join('bayesian_tuning_v4',
'customer_seg_final', f"trial_{trial_id}", "trial_history.csv")

    if os.path.exists(history_path):
        history_df = pd.read_csv(history_path)
        hps = trial.hyperparameters.values

        # --- Row 1: Loss Plot ---
        ax_loss = axes[0, i]
        # Labeling Training and Validation explicitly
        ax_loss.plot(history_df['loss'], label='Train Loss',
color='#1f77b4', linewidth=1.5)
        ax_loss.plot(history_df['val_loss'], label='Val Loss',
color='#ff7f0e', linestyle='--', linewidth=1.5)

        ax_loss.set_title(f"Rank {i+1} (ID: {trial_id})\n
n{hps.get('optimizer_name').upper()} | LR:
{hps.get('learning_rate'):.1e}", fontsize=11, fontweight='bold')
        ax_loss.set_ylabel('Loss')
        ax_loss.grid(True, alpha=0.3)
        if i == 0: ax_loss.legend() # Legend for the first column

        # --- Row 2: Accuracy Plot ---
        ax_acc = axes[1, i]
        # Added labels here so they can be identified in a legend

```



```

ax_acc.plot(history_df['accuracy'], label='Train Acc',
color='#2ca02c', linewidth=1.5)
ax_acc.plot(history_df['val_accuracy'], label='Val Acc',
color='#d62728', linestyle='--', linewidth=1.5)

ax_acc.set_ylabel('Accuracy')
ax_acc.set_xlabel('Epochs')

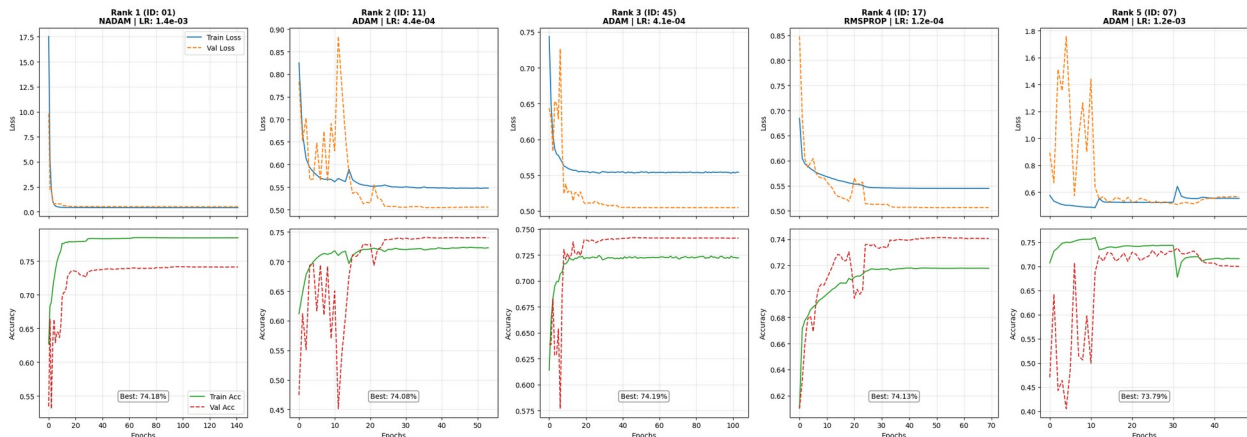
# Annotate best validation accuracy
best_val = history_df['val_accuracy'].max()
ax_acc.annotate(f'Best: {best_val:.2%}', xy=(0.5, 0.1),
xycoords='axes fraction',
ha='center', fontsize=10,
bbox=dict(boxstyle="round", fc="w", alpha=0.5))

ax_acc.grid(True, alpha=0.3)
if i == 0: ax_acc.legend() # Added legend to first column of
Accuracy row

plt.suptitle(f"Top {top_n} Bayesian Optimization Trials: Convergence
Analysis", fontsize=16, y=0.98)
plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.show()

```

Top 5 Bayesian Optimization Trials: Convergence Analysis



```

print("\nExtracting the best epoch from the CSV logs...")

# 1. Get the best trial object from the tuner
best_trial = tuner.oracle.get_best_trials(num_trials=1)[0]
best_trial_id = best_trial.trial_id

# 2. Reconstruct the path to that specific trial's CSV file
best_trial_csv_path = os.path.join(
    tuner.directory,

```

```

        tuner.project_name,
        f"trial_{best_trial_id}",
        "trial_history.csv"
    )

# 3. Read the CSV using Pandas
history_df = pd.read_csv(best_trial_csv_path)

# 4. Find the row with the minimum validation loss
# Note: CSVLogger logs epochs starting at 0, so we add 1 for the
actual step count
best_epoch_step = history_df.loc[history_df['val_loss'].idxmin(),
    'epoch'] + 1

print(f"--> Best trial ID: {best_trial_id}")
print(f"--> Lowest validation loss achieved at epoch:
{best_epoch_step}")

# Rebuild the best model architecture
best_hps = tuner.get_best_hyperparameters(num_trials=1)[0]
final_model = tuner.hypermodel.build(best_hps)

# Combine train and validation data
X_final_train = np.concatenate((X_train_resampled, X_val), axis=0)
y_final_train = np.concatenate((y_train_resampled, y_val), axis=0)

# Apply the 20% heuristic for the larger dataset
optimal_epochs = int(best_epoch_step * 1.20)

# Set up the final dataset pipeline
BATCH_SIZE = best_hps.get('batch_size') if 'batch_size' in
best_hps.values else 32
final_train_ds = (
    tf.data.Dataset.from_tensor_slices((X_final_train, y_final_train))
        .shuffle(len(X_final_train))
        .batch(BATCH_SIZE)
        .prefetch(tf.data.AUTOTUNE)
)

print(f"\nRetraining the final model on the full combined dataset for
{optimal_epochs} epochs...")

# Train without EarlyStopping so it hits the exact calculated target
final_history = final_model.fit(
    final_train_ds,
    epochs=optimal_epochs,
    verbose=1
)

```

```
# Save the finalized model
final_model.save('final_customer_segmentation_model.keras')
print("\nModel retraining complete and saved to disk.")
```

Extracting the best epoch from the CSV logs...

--> Best trial ID: 01

--> Lowest validation loss achieved at epoch: 127

Retraining the final model on the full combined dataset for 152 epochs...

Epoch 1/152

2946/2946 [=====] - 20s 5ms/step - loss: 12.5757 - accuracy: 0.6036

Epoch 2/152

2946/2946 [=====] - 16s 5ms/step - loss: 4.0851 - accuracy: 0.6674

Epoch 3/152

2946/2946 [=====] - 16s 5ms/step - loss: 2.1303 - accuracy: 0.6868

Epoch 4/152

2946/2946 [=====] - 16s 5ms/step - loss: 1.2842 - accuracy: 0.7028

Epoch 5/152

2946/2946 [=====] - 16s 5ms/step - loss: 0.8795 - accuracy: 0.7153

Epoch 6/152

2946/2946 [=====] - 16s 5ms/step - loss: 0.6547 - accuracy: 0.7284

Epoch 7/152

2946/2946 [=====] - 16s 5ms/step - loss: 0.5233 - accuracy: 0.7450

Epoch 8/152

2946/2946 [=====] - 15s 5ms/step - loss: 0.4942 - accuracy: 0.7528

Epoch 9/152

2946/2946 [=====] - 15s 5ms/step - loss: 0.4915 - accuracy: 0.7541

Epoch 10/152

2946/2946 [=====] - 14s 5ms/step - loss: 0.4882 - accuracy: 0.7561

Epoch 11/152

2946/2946 [=====] - 14s 5ms/step - loss: 0.4798 - accuracy: 0.7588

Epoch 12/152

2946/2946 [=====] - 14s 5ms/step - loss: 0.4733 - accuracy: 0.7636

Epoch 13/152

2946/2946 [=====] - 14s 5ms/step - loss:

0.4706 - accuracy: 0.7632
Epoch 14/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4677 - accuracy: 0.7649
Epoch 15/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4651 - accuracy: 0.7656
Epoch 16/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4638 - accuracy: 0.7664
Epoch 17/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4619 - accuracy: 0.7664
Epoch 18/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4612 - accuracy: 0.7673
Epoch 19/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4595 - accuracy: 0.7685
Epoch 20/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4584 - accuracy: 0.7679
Epoch 21/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4576 - accuracy: 0.7693
Epoch 22/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4569 - accuracy: 0.7684
Epoch 23/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4575 - accuracy: 0.7672
Epoch 24/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4554 - accuracy: 0.7688
Epoch 25/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4546 - accuracy: 0.7679
Epoch 26/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4544 - accuracy: 0.7680
Epoch 27/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4538 - accuracy: 0.7688
Epoch 28/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4551 - accuracy: 0.7684
Epoch 29/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4528 - accuracy: 0.7680

Epoch 30/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4528 - accuracy: 0.7691
Epoch 31/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4520 - accuracy: 0.7684
Epoch 32/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4508 - accuracy: 0.7701
Epoch 33/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4501 - accuracy: 0.7696
Epoch 34/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4509 - accuracy: 0.7699
Epoch 35/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4502 - accuracy: 0.7705
Epoch 36/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4502 - accuracy: 0.7686
Epoch 37/152
2946/2946 [=====] - 17s 6ms/step - loss:
0.4498 - accuracy: 0.7708
Epoch 38/152
2946/2946 [=====] - 17s 6ms/step - loss:
0.4494 - accuracy: 0.7700
Epoch 39/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4529 - accuracy: 0.7700
Epoch 40/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4502 - accuracy: 0.7707
Epoch 41/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4489 - accuracy: 0.7709
Epoch 42/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4468 - accuracy: 0.7698
Epoch 43/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4490 - accuracy: 0.7713
Epoch 44/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4483 - accuracy: 0.7703
Epoch 45/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4473 - accuracy: 0.7713
Epoch 46/152

```
2946/2946 [=====] - 15s 5ms/step - loss:
0.4471 - accuracy: 0.7718
Epoch 47/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4488 - accuracy: 0.7722
Epoch 48/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4472 - accuracy: 0.7713
Epoch 49/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4481 - accuracy: 0.7702
Epoch 50/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4473 - accuracy: 0.7715
Epoch 51/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4464 - accuracy: 0.7713
Epoch 52/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4436 - accuracy: 0.7713
Epoch 53/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4434 - accuracy: 0.7716
Epoch 54/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4463 - accuracy: 0.7720
Epoch 55/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4438 - accuracy: 0.7713
Epoch 56/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4440 - accuracy: 0.7706
Epoch 57/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4443 - accuracy: 0.7714
Epoch 58/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4485 - accuracy: 0.7720
Epoch 59/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4465 - accuracy: 0.7725
Epoch 60/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4440 - accuracy: 0.7729
Epoch 61/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4423 - accuracy: 0.7719
Epoch 62/152
2946/2946 [=====] - 15s 5ms/step - loss:
```

0.4445 - accuracy: 0.7727
Epoch 63/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4435 - accuracy: 0.7730
Epoch 64/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4421 - accuracy: 0.7715
Epoch 65/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4453 - accuracy: 0.7713
Epoch 66/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4464 - accuracy: 0.7721
Epoch 67/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4449 - accuracy: 0.7716
Epoch 68/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4431 - accuracy: 0.7727
Epoch 69/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4443 - accuracy: 0.7729
Epoch 70/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4434 - accuracy: 0.7736
Epoch 71/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4436 - accuracy: 0.7721
Epoch 72/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4418 - accuracy: 0.7727
Epoch 73/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4418 - accuracy: 0.7726
Epoch 74/152
2946/2946 [=====] - 14s 5ms/step - loss:
0.4426 - accuracy: 0.7715
Epoch 75/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4426 - accuracy: 0.7722
Epoch 76/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4421 - accuracy: 0.7721
Epoch 77/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4444 - accuracy: 0.7723
Epoch 78/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4453 - accuracy: 0.7724

Epoch 79/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4415 - accuracy: 0.7722
Epoch 80/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4422 - accuracy: 0.7732
Epoch 81/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4417 - accuracy: 0.7734
Epoch 82/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4419 - accuracy: 0.7730
Epoch 83/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4411 - accuracy: 0.7722
Epoch 84/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4408 - accuracy: 0.7733
Epoch 85/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4432 - accuracy: 0.7719
Epoch 86/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4426 - accuracy: 0.7721
Epoch 87/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4397 - accuracy: 0.7732
Epoch 88/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4412 - accuracy: 0.7733
Epoch 89/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4414 - accuracy: 0.7727
Epoch 90/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4507 - accuracy: 0.7714
Epoch 91/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4453 - accuracy: 0.7729
Epoch 92/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4404 - accuracy: 0.7733
Epoch 93/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4395 - accuracy: 0.7721
Epoch 94/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4397 - accuracy: 0.7739
Epoch 95/152


```
2946/2946 [=====] - 16s 5ms/step - loss:
0.4399 - accuracy: 0.7721
Epoch 96/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4414 - accuracy: 0.7735
Epoch 97/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4392 - accuracy: 0.7747
Epoch 98/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4395 - accuracy: 0.7737
Epoch 99/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4401 - accuracy: 0.7729
Epoch 100/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4396 - accuracy: 0.7736
Epoch 101/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4392 - accuracy: 0.7741
Epoch 102/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4384 - accuracy: 0.7726
Epoch 103/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4387 - accuracy: 0.7739
Epoch 104/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4469 - accuracy: 0.7705
Epoch 105/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4410 - accuracy: 0.7739
Epoch 106/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4430 - accuracy: 0.7738
Epoch 107/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4404 - accuracy: 0.7731
Epoch 108/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4441 - accuracy: 0.7738
Epoch 109/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4404 - accuracy: 0.7734
Epoch 110/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4414 - accuracy: 0.7744
Epoch 111/152
2946/2946 [=====] - 16s 5ms/step - loss:
```

```
0.4405 - accuracy: 0.7740
Epoch 112/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4420 - accuracy: 0.7728
Epoch 113/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4405 - accuracy: 0.7738
Epoch 114/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4387 - accuracy: 0.7730
Epoch 115/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4392 - accuracy: 0.7735
Epoch 116/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4410 - accuracy: 0.7732
Epoch 117/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4383 - accuracy: 0.7736
Epoch 118/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4395 - accuracy: 0.7741
Epoch 119/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4403 - accuracy: 0.7718
Epoch 120/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4396 - accuracy: 0.7735
Epoch 121/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4398 - accuracy: 0.7737
Epoch 122/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4379 - accuracy: 0.7735
Epoch 123/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4395 - accuracy: 0.7738
Epoch 124/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4374 - accuracy: 0.7743
Epoch 125/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4376 - accuracy: 0.7737
Epoch 126/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4383 - accuracy: 0.7736
Epoch 127/152
2946/2946 [=====] - 15s 5ms/step - loss:
0.4390 - accuracy: 0.7747
```

Epoch 128/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4383 - accuracy: 0.7734
Epoch 129/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4373 - accuracy: 0.7732
Epoch 130/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4364 - accuracy: 0.7739
Epoch 131/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4388 - accuracy: 0.7735
Epoch 132/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4409 - accuracy: 0.7739
Epoch 133/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4371 - accuracy: 0.7730
Epoch 134/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4389 - accuracy: 0.7744
Epoch 135/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4383 - accuracy: 0.7746
Epoch 136/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4384 - accuracy: 0.7739
Epoch 137/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4377 - accuracy: 0.7743
Epoch 138/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4378 - accuracy: 0.7728
Epoch 139/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4386 - accuracy: 0.7740
Epoch 140/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4362 - accuracy: 0.7748
Epoch 141/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4386 - accuracy: 0.7730
Epoch 142/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4367 - accuracy: 0.7742
Epoch 143/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4405 - accuracy: 0.7730
Epoch 144/152

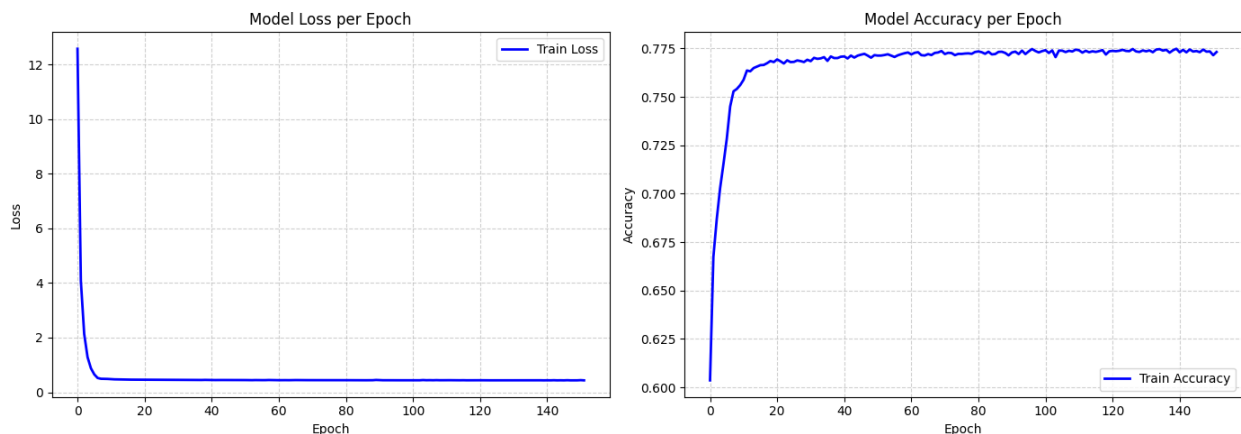
```

2946/2946 [=====] - 16s 5ms/step - loss:
0.4361 - accuracy: 0.7744
Epoch 145/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4384 - accuracy: 0.7734
Epoch 146/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4359 - accuracy: 0.7736
Epoch 147/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4411 - accuracy: 0.7731
Epoch 148/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4361 - accuracy: 0.7744
Epoch 149/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4363 - accuracy: 0.7733
Epoch 150/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4368 - accuracy: 0.7734
Epoch 151/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4434 - accuracy: 0.7715
Epoch 152/152
2946/2946 [=====] - 16s 5ms/step - loss:
0.4358 - accuracy: 0.7732

```

Model retraining complete and saved to disk.

```
plot_training_history(final_history)
```



```

optimized_metrics = evaluate_multiclass_model(final_model,
X_train_resampled, y_train_resampled, 2)

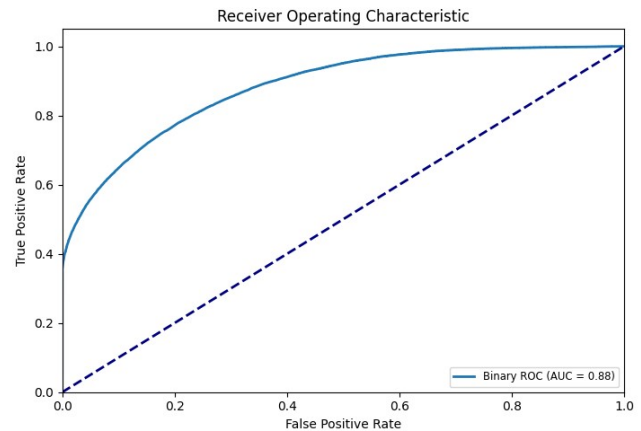
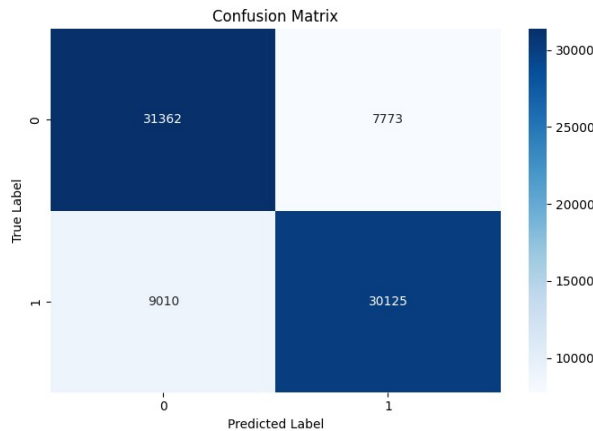
```

```

2446/2446 [=====] - 3s 980us/step
Accuracy: 0.7856

```

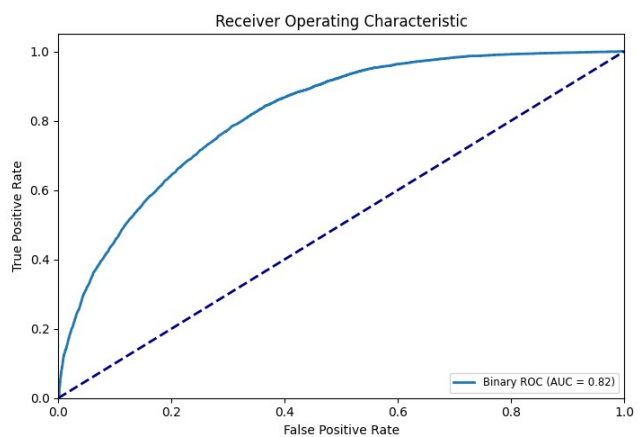
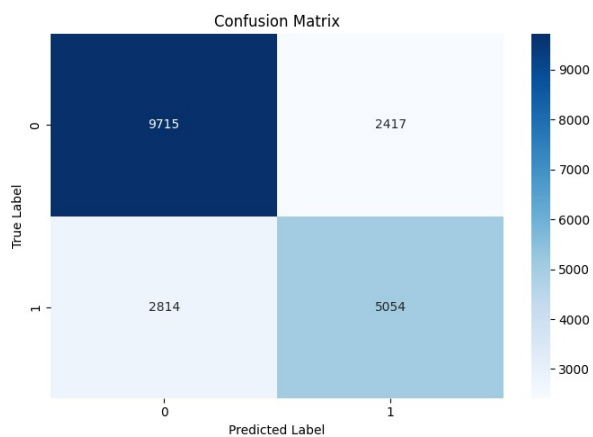
Precision (Macro): 0.7859
Recall (Macro): 0.7856
F1-score (Macro): 0.7855



```
optimized_metrics = evaluate_multiclass_model(final_model, X_test,  
y_test, 2)
```

625/625 [=====] - 1s 1ms/step

Accuracy: 0.7385
Precision (Macro): 0.7259
Recall (Macro): 0.7216
F1-score (Macro): 0.7234



Saving Optimized Results

```
# Convert the comprehensive cross-validation results to a DataFrame  
# cv_results_df = pd.DataFrame(search_result.cv_results_)  
  
# Sort by rank to ensure the best performing models are at the top  
# cv_results_df = cv_results_df.sort_values('rank_test_score')
```

```

# Save Training History to CSV
history_df = pd.DataFrame(final_history.history)
history_df.to_csv('baseline_training_history.csv',
index_label='epoch')

# Save the results to a CSV file for reporting
# cv_results_df.to_csv('random_search_summary_results.csv',
index=False)

# Save Training History to CSV
# history_df = pd.DataFrame(search_result.best_estimator_.history_)
# history_df.to_csv('baseline_training_history.csv',
index_label='epoch')

# Save Evaluation Metrics to CSV
optimized_metrics.to_csv('baseline_model_metrics.csv', index=False)

```

Plotting Influence of Parameters on Results

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import numpy as np

def plot_tuner_analysis_refined(tuner):
    # Extract trial results into a structured DataFrame
    trials =
tuner.oracle.get_best_trials(num_trials=len(tuner.oracle.trials))
    data = []
    for trial in trials:
        trial_data = trial.hyperparameters.values.copy()
        trial_data['score'] = trial.score
        data.append(trial_data)

    results_df = pd.DataFrame(data)
    param_cols = [col for col in results_df.columns if col != 'score']

    num_params = len(param_cols)
    num_cols = 3
    num_rows = (num_params + num_cols - 1) // num_cols

    fig, axes = plt.subplots(num_rows, num_cols, figsize=(18, 5 *
num_rows))
    axes = axes.flatten()

    for i, col in enumerate(param_cols):
        display_name = col.replace('model__', '')

```

```

        # Enforce 3 decimal places on the Y-axis (Validation Accuracy)
axes[i].yaxis.set_major_formatter(mtick.FormatStrFormatter('%0.3f'))

        if col in ['weight_decay', 'learning_rate']:
            sns.scatterplot(x=col, y='score', data=results_df,
ax=axes[i], alpha=0.6)
            axes[i].set_xscale('log')
            # For log scales, scientific notation is maintained for
            readability
        else:
            # Round continuous numeric values to 3 decimals for clean
            categorical labels
            plot_df = results_df.copy()
            if plot_df[col].dtype in [np.float64, np.float32]:
                plot_df[col] = plot_df[col].round(3)

            sns.boxplot(x=col, y='score', data=plot_df, ax=axes[i],
hue=col, legend=False)
            axes[i].tick_params(axis='x', rotation=45)

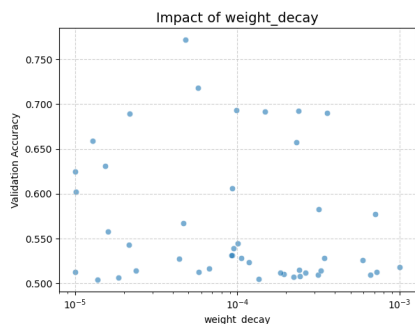
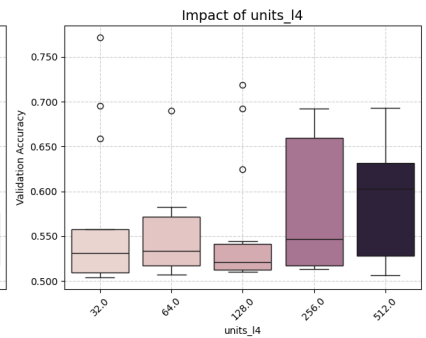
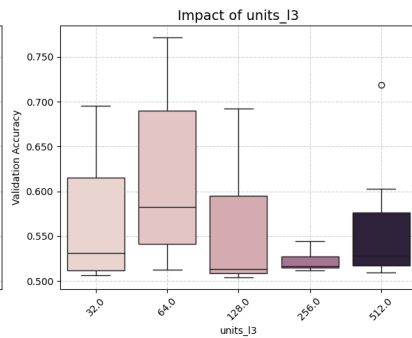
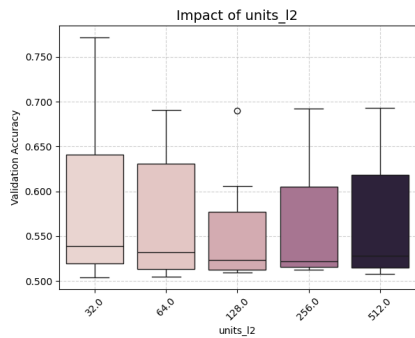
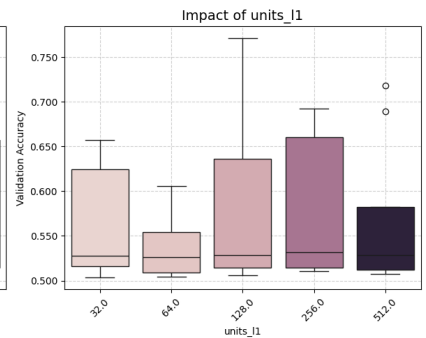
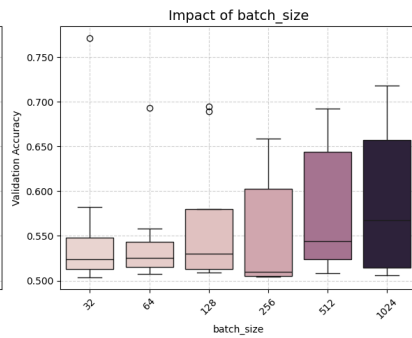
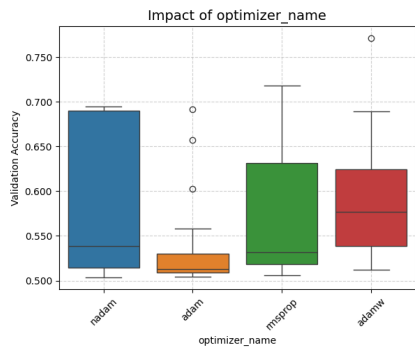
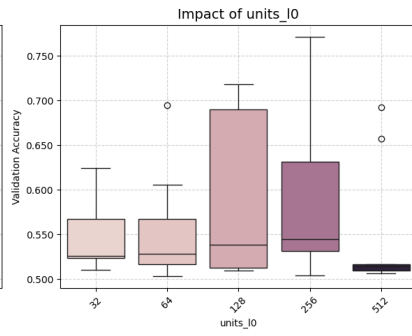
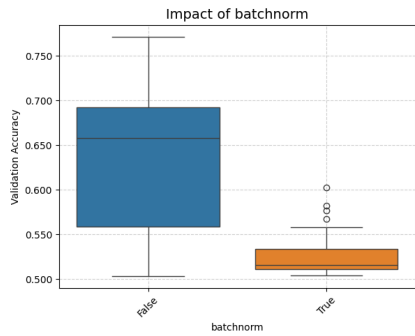
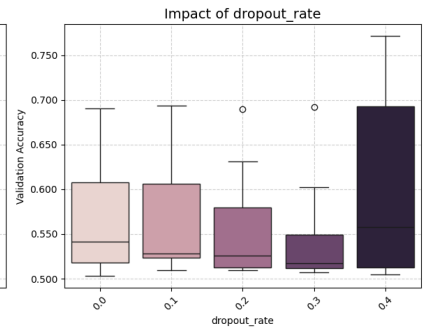
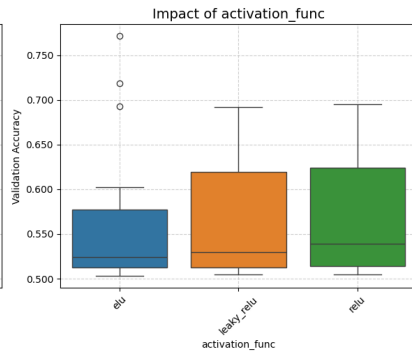
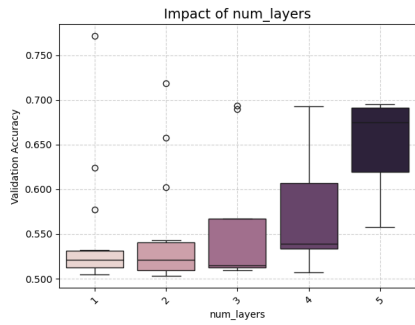
            axes[i].set_title(f'Impact of {display_name}', fontsize=14)
            axes[i].set_xlabel(display_name)
            axes[i].set_ylabel('Validation Accuracy')
            axes[i].grid(True, linestyle='--', alpha=0.6)

        for j in range(i + 1, len(axes)):
            fig.delaxes(axes[j])

        plt.tight_layout()
        plt.show()

# Execute the visualization
plot_tuner_analysis_refined(tuner)

```



End of Baseline and Optimization of ANN